

DSAA3073: Theories in Data Science

Week 8: Online Learning and Prediction with Expert Advice

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-07-15

Explorative projection

DSAA3073: Theories in Data Science

Week 8: Section 1: Introduction & Online Learning Framework (Cognitive Journey)

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-07-15

Explorative projection

Introduction: From Batch to Online Learning

Backbone message: This week reveals a fundamental principle: **learning under adversarial uncertainty is possible through regret minimization**. We'll discover how shifting from absolute performance (generalization error) to relative performance (regret) enables learning even when the world actively works against us—no distributional assumptions required.

The Question That Changes Everything

🧐 Student Question

“Why compare to the best expert we could have chosen in hindsight? That seems unfair—if I knew which expert would be best, I wouldn't need an algorithm at all!”

This is Crisis 7: **The Hindsight Comparison Shock**. It's the question that unlocks the entire framework.

Let's understand why this seems wrong, and why it's actually the only sensible approach.

The Setup: Expert Advice Without a Safety Net

You're managing a portfolio. You have access to $n = 1000$ financial analysts, each providing daily buy/sell recommendations. Over $T = 10000$ trading days, you follow an algorithm and incur total loss $L_{\text{algo}} = 1500$ (measured in thousands of dollars).

At the end, you look back and discover: **Analyst 347 had the best track record**, with total loss $L^* = 1300$.

Your algorithm's regret is: $R_T = 1500 - 1300 = 200$.

The student's reaction: “This is absurd! I'm being compared to someone who was revealed to be best **after the fact**. If you told me on Day 1 that Analyst 347 would be best, I'd just follow them and get loss 1300. But I didn't have that information! This comparison seems impossibly unfair.”

Pause and Think

Question: Is this comparison actually unfair? What alternatives do we have?

Hint: Consider: what if we compared to the **average** analyst instead of the best one?

Answer:

Or: what if we just tried to minimize our absolute loss without any comparison?

The Direct Approach: Apply PAC Learning Online

Let's try the natural approach: treat this like batch learning, but update our model sequentially.

Strategy 1: Maintain a Fixed Hypothesis

Idea: Train a hypothesis h on past data, use it for future predictions, update periodically.

From Week 1-7: We know that with i.i.d. samples from distribution $\mathcal{D}$, if $m \geq m_{\text{PAC}}(\varepsilon, \delta, d)$ where d is the VC dimension:

$$\text{Prob}\left[R(h_S) - \min_{h \in \mathcal{H}} R(h) \leq \varepsilon\right] \geq 1 - \delta \quad (1)$$

So we could:

1. Collect data from rounds $1, 2, \dots, t$
2. Train hypothesis $h_t = \text{ERM}(\{z_1, \dots, z_t\})$
3. Use h_t for predictions in round $t + 1$
4. Update when we get new data

Seems reasonable: We're using the empirical risk minimization principle, which we know works when data is i.i.d.

Let's test this with concrete numbers.

Why This Fails: The I.I.D. Assumption Collapses

⚠ Crisis

The Fatal Dependency

PAC learning requires i.i.d. samples from a fixed distribution $\mathcal{D}$.

In online learning with an adversarial environment:

- The “distribution” can change arbitrarily between rounds
- The adversary can observe our hypothesis h_t and choose the next outcome y_{t+1} to make h_t wrong
- The data is **not** independent: y_{t+1} depends on our choice of h_t

Without i.i.d., generalization error is undefined.

Let's see this break down numerically.

Concrete Failure Example

Setup: Binary classification, $\mathcal{X} = \{0, 1\}$, $\mathcal{Y} = \{0, 1\}$. Two hypotheses:

- h_1 : always predict 0
- h_2 : always predict 1

Adversarial strategy:

- Observe which hypothesis the learner is using
- Choose outcome to make it wrong

What happens:

Round	Learner's h_t	Adversary's y_t	Loss	Empirical Risk
1	h_1 (predict 0)	1	1	$\hat{R}_S(h_1) = 1$
2	h_2 (predict 1)	0	1	$\hat{R}_S(h_2) = \frac{1}{2}, \hat{R}_S(h_1) = \frac{1}{2}$
3	Switch to h_2	0	1	$\hat{R}_S(h_2) = \frac{2}{3}, \hat{R}_S(h_1) = \frac{1}{3}$
4	Switch to h_1	1	1	$\hat{R}_S(h_1) = \frac{2}{4}, \hat{R}_S(h_2) = \frac{1}{4}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

The adversary doesn't need to be omniscient. They just need to observe which hypothesis we're currently using and choose outcomes accordingly. This is realistic in many settings (e.g., spam filters vs. spammers, security systems vs. attackers).

Result: The learner makes a mistake **every single round**, regardless of which hypothesis they choose!

- Total learner loss after T rounds: $L_{\text{learner}} = T$
- Best single hypothesis: $L^* = \frac{T}{2}$ (whichever one matches the adversary's majority outcome)
- Gap: $\frac{T}{2}$ mistakes—**linear** in T

Why this destroys PAC learning:

- Generalization error $R(h)$ is undefined without a distribution
- Empirical risk $\hat{R}(h)$ keeps fluctuating as adversary adapts
- We can't appeal to uniform convergence—it requires i.i.d. samples

The Structural Obstacle: No Distribution Means No Generalization

Let's understand **why** PAC learning fundamentally cannot work in adversarial settings.

Theorem Landscape

The Impossibility Result

In PAC learning, the guarantee is: $R(h_S) \approx \hat{R}(h_S)$ with high probability over **i.i.d. samples** from distribution $\mathcal{D}$.

But in online learning:

- $R(h) = \mathbb{E}_{x,y \sim \mathcal{D}}[\ell(h, (x, y))]$ requires a distribution $\mathcal{D}$
- No distribution exists when the adversary chooses outcomes arbitrarily
- Therefore $R(h)$ is **undefined**

Conclusion: Generalization error is the wrong metric for adversarial online learning.

What Goes Wrong Mathematically

PAC learning relies on the law of large numbers. For a fixed hypothesis h : $\hat{R}(h) = \frac{1}{m} \sum_{i=1}^m \ell(h, z_i) \xrightarrow{m \rightarrow \infty} \mathbb{E}_{z \sim \mathcal{D}}[\ell(h, z)] = R(h)$

This convergence requires:

1. The z_i are i.i.d. from some distribution $\mathcal{D}$
2. The distribution $\mathcal{D}$ is **fixed** (doesn't depend on h or the algorithm)

In adversarial online learning:

1. The outcomes y_t can depend on past predictions (not independent)
2. No fixed distribution exists (adversary chooses adaptively)

Mathematically: We're trying to compute an expectation $E[\ell]$, but there's no probability distribution to take the expectation over!

🤔 Student Question

“If there's no distribution, how can learning work at all? Don't we need some statistical regularity to learn from data?”

This is Crisis 9: **Learning Without Distributional Assumptions**. It seems impossible—and yet, it's not.

The key insight: **we need a different performance metric.**

The Transformation Device: Regret as Relative Performance

Here's the crucial conceptual shift that makes online learning possible.

💡 Aha Moment

The Resolution: Compare Relative Performance, Not Absolute

Instead of asking “How well does my algorithm perform?” (absolute performance, requires distribution), ask:

“How much worse is my algorithm than the best single expert in hindsight?” (relative performance, no distribution needed)

This is called **regret**:

$$R_T = \sum_{t=1}^T \ell(\text{algorithm}, z_t) - \min_{i \in [n]} \sum_{t=1}^T \ell(\text{expert } i, z_t) \quad (2)$$

Key property: Regret is defined using only the **observed sequence** $(z_1, \dots, z_T)$ —no distribution required!

Why This Works: The Benchmark Is Internal

The genius of regret is that the benchmark (best expert) and the algorithm are both evaluated on **the same sequence** of outcomes.

PAC learning:

- Algorithm sees training data $S: (x_1, y_1), \dots, (x_m, y_m)$
- Tested on **different** data from same distribution: $(x'_1, y'_1), \dots$
- Need distribution to relate training error to test error

Online learning:

- Algorithm makes predictions $\hat{y}_1, \dots, \hat{y}_T$

- Best expert makes predictions on **same outcomes** $y_1, \dots, y_T$
- Both evaluated on identical sequence—no distribution needed!

The comparison is **internal** to the observed sequence, not **external**

The term “regret” comes from decision theory: it’s the “regret” you feel for not having chosen the best action in hindsight. In online learning, it’s a technical term for the performance gap.

Formal Definition

Definition: Regret

For an online learning algorithm $\mathcal{A}$ running for T rounds against n experts, the **regret** is:

$$R_T = \sum_{t=1}^T \ell_t(\mathcal{A}) - \min_{i=1, \dots, n} \sum_{t=1}^T \ell_t(i) \quad (3)$$

where:

- $\ell_t(\mathcal{A})$ is the loss incurred by the algorithm in round t
- $\ell_t(i)$ is the loss incurred by expert i in round t
- The minimum is over all n experts

Interpretation: Regret measures how many more mistakes (or how much more loss) the algorithm makes compared to the best single expert in hindsight.

Crisis 5: The Sublinear Regret Paradox

Now we can address the opening question: why is comparing to hindsight reasonable?

Let’s look at concrete numbers that reveal the paradox.

⚠ Crisis

The Paradox of Growing Regret

Setup: $n = 1000$ experts, $T = 10000$ rounds. Your algorithm achieves:

$$R_T = 200 \quad (4)$$

Student’s reaction: “I made 200 more mistakes than the best expert! That’s terrible! Why not just pick one expert randomly at the start and stick with them?”

The Numerical Comparison

Consider $T = 10000$ rounds with $n = 1000$ experts. Random expert selection has expected loss equal to the average expert (5000 mistakes if average expert makes 50% errors), but with high variance—you might pick the worst expert (7000 mistakes). Meanwhile, MWU guarantees $R_T \leq 2\sqrt{T \ln n} \approx 166$, so your loss is at most $L^* + 166$ where L^* is the best expert’s loss (e.g., ≤ 3166 if $L^* = 3000$). MWU pays only $\approx 1.7\%$ overhead to avoid the risk of picking a terrible expert.

Why “200 Mistakes Behind” Is Actually Amazing

Let's break down what $R_T = 200$ really means:

Absolute regret: 200 total mistakes

- Sounds like: "I'm 200 mistakes behind"
- Feels like: "I'm learning nothing"

Average regret per round: $\frac{R_T}{T} = \frac{200}{10000} = 0.02$

- Means: 0.02 extra mistakes per round
- As $T \rightarrow \infty$: $\frac{R_T}{T} \rightarrow 0$

Interpretation:

- After 10,000 rounds, you've essentially identified the best expert
- You're making mistakes at almost exactly their rate
- The 200 mistakes are the **learning cost**—the price of figuring out who's best
- This cost is **sublinear**: $O(\sqrt{T})$ grows much slower than T

💡 Aha Moment

Why Sublinear Regret Means Success

If $R_T = o(T)$ (grows slower than T), then:

$$\frac{\text{Your total loss}}{T} = \frac{L^* + R_T}{T} = \frac{L^*}{T} + \frac{R_T}{T} \rightarrow \frac{L^*}{T} \quad (5)$$

Your **average loss per round** converges to the best expert's average loss!

You're learning to match the best expert's performance, despite:

- Not knowing who's best in advance
- Facing an adversarial environment
- Having no distributional assumptions

Concrete Scaling: How Regret Grows

Let's see how regret scales with T :

Rounds T	Regret $R_T = 2\sqrt{T \ln n}$	Average R_T/T	Overhead %
100	≈ 33	0.33	33%
1,000	≈ 105	0.105	10.5%
10,000	≈ 166	0.017	1.7%
100,000	≈ 525	0.005	0.5%
1,000,000	$\approx 1,661$	0.002	0.2%

Observation: As T grows:

- Absolute regret R_T grows ($\approx \sqrt{T}$)
- Average regret R_T/T shrinks ($\approx 1/\sqrt{T}$)
- You get arbitrarily close to best expert's performance

This is why $O(\sqrt{T})$ regret is considered **optimal** in adversarial online learning!

Crisis 7 Revisited: Why Hindsight Comparison Is Necessary

Why compare to the best expert in hindsight? Alternative benchmarks fail: comparing to the **average expert** is too easy (trivial even when all experts are bad), comparing to a **Bayes-optimal classifier** requires distributional assumptions we don't have, and comparing to the **best adaptive strategy** is impossible (the adversary can always win). The best single expert in hindsight is the unique benchmark that is both achievable ($O(\sqrt{T})$ regret is possible) and meaningful (non-trivial when experts are diverse), while remaining distribution-free. We're not trying to beat oracle knowledge—we're matching the best fixed strategy despite not knowing which one is best in advance.

The Resolution: Regret Enables Distribution-Free Learning

Let's synthesize what we've discovered.

Aha Moment

How Learning Works Without Distributions

The adversary can make outcomes arbitrarily difficult, but they cannot make us worse than the best expert by more than $O(\sqrt{T})$.

Key insight:

- If adversary makes all outcomes 0, best expert (always predicts 0) makes 0 mistakes, we make $\leq 0 + O(\sqrt{T})$
- If adversary makes outcomes random, best expert makes $\approx \frac{T}{2}$ mistakes, we make $\leq \frac{T}{2} + O(\sqrt{T})$
- If adversary makes outcomes adversarial, best expert makes L^* mistakes, we make $\leq L^* + O(\sqrt{T})$

Regret is relative, not absolute. We track whoever is best, regardless of how good or bad they are.

The Mathematical Guarantee

For Multiplicative Weights Update (MWU), we'll prove:

$$R_T \leq 2\sqrt{T \ln n} \quad (6)$$

This holds:

- For **any** sequence of outcomes $(y_1, \dots, y_T)$
- Against **any** adversarial strategy
- Without **any** distributional assumptions
- For **any** number of experts n

This is remarkable: Learning is possible in the worst-case adversarial setting!

Framework Definition: Online Learning Protocol

Now we can formally define the online learning framework.

For most of this week, we focus on bounded losses $\ell \in [0, 1]$. Binary losses are a special case where $\ell \in \{0, 1\}$.

Definition: Online Learning Protocol

The online learning game proceeds in T rounds. In each round $t = 1, 2, \dots, T$:

1. **Each expert i** makes a prediction $\hat{y}_i^t \in \mathcal{Y}$
2. **Learner** observes predictions $\{\hat{y}_1^t, \dots, \hat{y}_n^t\}$
3. **Learner** chooses a prediction $\hat{y}_t \in \mathcal{Y}$ (possibly randomized)
4. **Environment** reveals the true outcome $y_t \in \mathcal{Y}$
5. **Learner and experts** incur losses based on y_t

Notation:

- $\ell_t(i) = \ell(\hat{y}_i^t, y_t)$: loss of expert i in round t
- $\ell_t(\text{algo}) = \ell(\hat{y}_t, y_t)$: loss of learner in round t

Goal: Minimize regret $R_T = \sum_{t=1}^T \ell_t(\text{algo}) - \min_{i=1}^n \sum_{t=1}^T \ell_t(i)$

Loss Functions

Definition: Loss Functions in Online Learning

Common loss functions:

1. Binary (0-1) loss:

$$\ell(\hat{y}, y) = \begin{cases} 0 & \text{if } \hat{y} = y \\ 1 & \text{if } \hat{y} \neq y \end{cases} \quad (7)$$

Used for classification. Loss $\in \{0, 1\}$.

2. Absolute loss:

$$\ell(\hat{y}, y) = |\hat{y} - y| \quad (8)$$

Used for regression. Continuous loss.

3. Squared loss:

$$\ell(\hat{y}, y) = (\hat{y} - y)^2 \quad (9)$$

Used for regression, more sensitive to outliers.

4. General bounded loss:

$$\ell(\hat{y}, y) \in [0, 1] \quad (10)$$

We often normalize losses to $[0, 1]$ for theoretical analysis.

Adversarial Models

Definition: Adversarial Models

Three models of increasing strength:

1. Oblivious adversary:

- Commits to entire sequence $(y_1, \dots, y_T)$ before the game starts
- Cannot adapt to learner's strategy
- **Weakest** model

2. Non-anticipating (adaptive) adversary:

- Chooses y_t after seeing learner's past actions $\hat{y}_1, \dots, \hat{y}_{t-1}$
- Cannot see current randomization used to produce $\hat{y}_t$
- **Standard** model for randomized algorithms

3. Fully adaptive adversary:

- Sees everything, including current randomization, before choosing y_t
- Can always make the sampled expert wrong
- **Strongest** model—no algorithm can achieve sublinear regret

Convention: When we say “adversarial online learning,” we mean the non-anticipating model unless stated otherwise.

Think of the non-anticipating adversary like a poker opponent: they see your past plays and your current face, but they don't see your hidden cards (randomization).

Comparison: PAC Learning vs. Online Learning

Let's consolidate the differences:

Aspect	PAC Learning	Online Learning
Data model	i.i.d. from distribution $\mathcal{D}$	Arbitrary/adversarial sequence
Performance metric	Generalization error $R(h) - \hat{R}(h)$	Regret R_T
Comparison	Test vs. train on same h	Algorithm vs. best expert on same sequence
Sample complexity	$m = O\left(\frac{d}{\epsilon^2} \log \frac{1}{\delta}\right)$	Time horizon T
Goal	$R(h_S) \approx \min_h R(h)$	$R_T = o(T)$ (sublinear)
Assumptions	Fixed distribution	None (worst-case)
Key tool	Uniform convergence	Multiplicative weights
Optimal rate	—	$O(\sqrt{T})$ regret

Connection

Unifying principle: Both frameworks ask “How can we compete with the best hypothesis/expert?”

- PAC: Best h achieves low $R(h)$, we want $R(h_S) \approx R(h^*)$
- Online: Best expert achieves low loss, we want $L_{\text{algo}} \approx L^* + o(T)$

The difference: PAC uses generalization (train vs. test), online uses regret (sequential competition).

Looking Ahead: The Multiplicative Weights Algorithm

We've established:

1. Online learning measures performance via regret (not generalization error)
2. Sublinear regret $O(\sqrt{T})$ is achievable and optimal
3. We compare to best expert in hindsight (strongest achievable benchmark)
4. No distributional assumptions needed

The remaining question: How do we actually achieve $O(\sqrt{T})$ regret?

The answer: **Multiplicative Weights Update (MWU)**.

Looking Ahead

Preview of What's Coming

The MWU algorithm:

- Maintains a weight $w_t(i)$ for each expert i
- Weights represent “credibility” based on past performance
- Updates multiplicatively: $w_{t+1}(i) = w_t(i) \cdot (1 - \eta \cdot \ell_t(i))$
- Samples expert proportional to weights (randomized prediction)

Key questions for Section 2:

- Why multiplicative updates rather than additive?
- Why randomize predictions rather than follow highest-weight expert?
- How do we choose the learning rate η ?
- Why does this achieve $O(\sqrt{T})$ regret?

Next section: We'll see the algorithm and prove the regret bound.

✓ Checkpoint

Checkpoint 3: Consider $n = 100$ experts, $T = 10000$ rounds. Algorithm achieves $R_T = 150$.

- (a) What is the average regret per round?
- (b) If the best expert made 2000 mistakes, how many did the algorithm make?
- (c) Is this better or worse than picking a random expert at the start?

Answers: (a) $\frac{R_T}{T} = \frac{150}{10000} = 0.015$ per round

(b) Algorithm made $\leq 2000 + 150 = 2150$ mistakes

(c) Much better! Random expert: expected ≈ 5000 mistakes (if average expert makes 50% errors). Algorithm: guaranteed ≤ 2150 mistakes. The algorithm pays only 150 extra mistakes to avoid the risk of picking a bad expert.

Summary: The Transformation from Batch to Online

🎯 Key Takeaway

Pattern 3 Resolution: Problem Transformation

Direct approach: Apply PAC learning online

- Maintain hypothesis, update with new data
- Fails: i.i.d. assumption violated, generalization error undefined

Structural obstacle: No distributional assumptions possible

- Adversary can adapt to our strategy
- Statistical learning theory requires distributions

Transformation device: Regret as performance metric

- Compare to best expert in hindsight (not Bayes-optimal)
- Defined using only observed sequence (no distribution)
- Sublinear regret $R_T = o(T)$ means learning succeeds

Resolution: Learning without distributions is possible

- Regret is relative, not absolute
- We track best expert despite not knowing who they are
- $O(\sqrt{T})$ regret is the price of online learning

Framework: Online learning protocol

- Sequential predictions, adversarial outcomes
- Goal: minimize regret against best expert
- Next: MWU algorithm achieves $O(\sqrt{T})$ regret

Pedagogical journey completed:

- Started with hindsight comparison objection (Crisis 7)
- Showed PAC approach fails structurally
- Transformed to regret-based comparison
- Resolved sublinear regret paradox (Crisis 5)
- Defined complete framework

Students now understand **why** online learning is different and **what** makes it possible.

The Halving Algorithm: The Perfect Expert Illusion

Student Question: Can We Achieve Logarithmic Mistakes?

Student Question

From the class: “We have $n = 100$ experts giving binary predictions. In the worst case, can we guarantee that we make at most $\log_2 100 \approx 7$ mistakes?”

This sounds almost too good to be true—logarithmic in the number of experts! If we had 1000 experts, we’d still only make about 10 mistakes. That’s incredibly efficient.

The intuition seems clear: if we’re smart about eliminating bad experts, the problem size shrinks exponentially fast. Binary search makes $\log n$ comparisons. Can we make $\log n$ mistakes?

Initial reaction: This question captures an exciting possibility. We’ve seen that naive strategies fail (Week 8’s “follow the currently best expert” can have linear regret). But what if we could systematically eliminate bad experts? Each mistake we make gives us information—maybe we can use it to cut the problem in half each time.

What makes this natural: Binary search achieves $O(\log n)$ comparisons by halving the search space. Decision trees have depth $\log_2 n$ for n leaves. Logarithmic scaling is the gold standard for divide-and-conquer algorithms.

Natural Approach: Aggressive Elimination

Strategy: Let’s think about what “perfect” means in the expert setting:

Definition: Perfect Expert Assumption

A **perfect expert** is one who **never makes a mistake** throughout all T rounds. We assume at least one such expert exists in our pool of n experts.

The elimination strategy:

If a perfect expert exists, then anyone who makes a mistake cannot be that perfect expert. So we should:

1. Start with all n experts as candidates
2. Each round: predict by **majority vote** among remaining experts
3. After seeing the outcome: **eliminate** all experts who were wrong

4. Continue with only the experts who could still be perfect

Thought Experiment

Thought experiment: You have 1000 fortune tellers predicting coin flips. You know exactly one has genuine psychic powers (never wrong), but the rest are frauds (random guessing).

- After seeing 1 wrong prediction, can someone still be the psychic? **No—eliminate them immediately.**
- After 10 rounds, roughly how many candidates remain? **About $\frac{1000}{2^{10}} \approx 1$**
- When we make a mistake via majority vote, at least half the remaining experts must be wrong (that's why majority lost). We eliminate at least half!

This suggests: **at most $\log_2 1000 \approx 10$ mistakes.**

Why this seems perfect:

- Clean elimination rule: one strike and you're out
- Exponential reduction: set size halves with each mistake
- Logarithmic bound: $\log_2 n$ mistakes looks optimal
- Simple to implement: no weights, no learning rate, just maintain a set

Let's formalize this as the **Halving Algorithm** and test it with real numbers.

The Halving Algorithm: Formal Description

Definition: Halving Algorithm

- Maintain set $S_t \subseteq \{1, \dots, n\}$ of experts that haven't made mistakes
- Initially: $S_1 = \{1, 2, \dots, n\}$
- Each round t :
 1. Predict by **majority vote** among experts in S_t
 2. Observe true outcome y_t
 3. Remove all experts $i \in S_t$ where prediction $\hat{y}_t^{(i)} \neq y_t$

The promised theorem:

Theorem: Halving Mistake Bound

If a perfect expert exists, the Halving Algorithm makes at most $\log_2 n$ mistakes.

The intuitive argument suggests $M \leq \log_2 n$ mistakes since each mistake eliminates at least half the experts. Let's test this.

Numerical Test: The Collapse

Setup: $n = 100$ experts, binary predictions $\in \{0, 1\}$. We'll simulate the first few rounds.

Example: Attempting Halving with $n=100$

Starting with 100 experts, suppose the adversary forces mistakes in each round:

- Round 1: 100 experts $\rightarrow$ mistake $\rightarrow$ 40 remain
- Round 2: 40 experts $\rightarrow$ mistake $\rightarrow$ 15 remain
- Round 3: 15 experts $\rightarrow$ mistake $\rightarrow$ 7 remain
- Round 4: 7 experts $\rightarrow$ mistake $\rightarrow$ 3 remain
- Round 5: 3 experts $\rightarrow$ mistake $\rightarrow$ 1 remains
- Round 6: 1 expert $\rightarrow$ mistake $\rightarrow$ **0 remain** — THE SET IS EMPTY

After 6 mistakes ($\approx \log_2 100$), we have $S_7 = \emptyset$. The algorithm has crashed—we can't make any more predictions.

The catastrophic failure: The bound $M \leq \log_2 n$ holds, but then the algorithm becomes **undefined** because all experts are eliminated.

Crisis: The Halving Collapse

⚠ Crisis

Crisis 1: The Halving Collapse

Setup: $n = 100$ experts. Halving Algorithm promises $\log_2 100 \approx 7$ mistakes.

Expected outcome: “We’ll make at most 7 mistakes and then identify the perfect expert. Clean logarithmic bound!”

Numerical reality:

- Round 1: 60 wrong $\rightarrow$ eliminate $\rightarrow$ 40 remain
- Round 2: 25 wrong $\rightarrow$ eliminate $\rightarrow$ 15 remain
- Round 3: 8 wrong $\rightarrow$ eliminate $\rightarrow$ 7 remain
- Round 4: 4 wrong $\rightarrow$ eliminate $\rightarrow$ 3 remain
- Round 5: 2 wrong $\rightarrow$ eliminate $\rightarrow$ 1 remains
- Round 6: 1 wrong $\rightarrow$ eliminate $\rightarrow$ **0 remain**
- Round 7: $S_7 = \emptyset \rightarrow$ **Algorithm undefined**

The gap:

- Expected: “7 mistakes then perfect performance”
- Reality: “6 mistakes then complete failure”

What went wrong: The $\log_2 n$ bound has a hidden requirement: **a perfect expert must exist**. Without a perfect expert:

- The bound doesn’t just fail to hold
- The algorithm doesn’t just make more mistakes
- The algorithm **crashes**—it becomes undefined

We eliminated everyone, including the best expert who only made 1 mistake in round 6. Now we have no one to follow.

Analysis of the failure:

The proof assumed: “Perfect expert never removed, so $|S_t| \geq 1$ always.” But if no perfect expert exists, then eventually everyone makes at least one mistake, and everyone gets eliminated.

Assumption	With perfect expert	Without perfect expert
Lower bound on $ S_t $	≥ 1 (perfect stays)	≥ 0 (everyone goes)
Mistake bound	$m \leq \log_2 n$	Undefined (set empties)
Algorithm status	Works perfectly	Crashes mid-stream

Concrete scenario where perfect expert doesn’t exist:

Consider $n = 4$ experts, $T = 10$ rounds:

- Expert 1: makes mistakes on rounds 3, 7
- Expert 2: makes mistakes on rounds 1, 5, 9
- Expert 3: makes mistakes on rounds 2, 6
- Expert 4: makes mistakes on rounds 4, 8, 10

No expert is perfect. Best expert (Expert 1) makes only 2 mistakes. But Halving will eventually eliminate everyone, including Expert 1.

Why Logarithmic is Impossible Without Perfect Expert

Information-theoretic argument:

Key Takeaway

To make at most m mistakes via majority vote:

- After m mistakes, at most $\frac{n}{2^m}$ experts remain (halving)
- Need at least 1 expert to remain: $\frac{n}{2^m} \geq 1$
- This gives: $m \leq \log_2 n$

But: if no perfect expert exists, best expert makes M^* mistakes. After M^* mistakes by the algorithm (tracking best expert closely), we might eliminate the best expert too.

The logarithmic bound is **impossibly optimistic** in realistic settings.

Reality check: In real applications:

- Stock market experts: all make wrong predictions sometimes
- Weather forecasters: all miss forecasts occasionally
- Medical diagnosis systems: all have errors
- Chess engines: all lose games against strong opponents

The “perfect expert” is a mathematical fiction. Halving Algorithm is theoretically beautiful but practically useless.

Aha Moment: Soft Elimination via Weights

Aha Moment

The key insight: Instead of **eliminating** bad experts (hard removal), we should **down-weight** them (soft removal). Expert makes mistake $\rightarrow$ reduce weight by factor $(1 - \eta)$. This gives the Weighted Majority Algorithm with weights $w_1(i) = 1$ and update $w_{t+1}(i) = (1 - \eta)w_t(i)$ when expert i errs. All weights stay positive, experts can recover, and regret grows as $O(\sqrt{T})$ without requiring a perfect expert.

Comparison of paradigms:

Property	Halving (hard elimination)	Weighted Majority (soft weights)
Mistake bound	$O(\log n)$	$O(\sqrt{T})$

Requires perfect expert?	Yes (else crashes)	No (handles any setting)
Expert recovery	Impossible (eliminated forever)	Possible (weight > 0 always)
Set becomes empty?	Yes (failure mode)	Never (weights stay positive)
Practical use	Theoretical only	Widely deployed

The trade-off:

- Halving: $O(\log n)$ mistakes but requires impossible assumption
- Weighted Majority: $O(\sqrt{T})$ mistakes but works in reality

Going from $\log n$ to $\sqrt{T}$ seems like a loss. But $\log 100 = 7$ under perfect expert vs “undefined” without perfect expert means we must pay the $\sqrt{T}$ price for robustness.

Formal Analysis of Halving (For Completeness)

Even though Halving fails in practice, its analysis is instructive. We formalize the algorithm and prove the bound—while highlighting the fatal assumption.

Definition: Halving Algorithm (Formal)

- Input: n experts making binary predictions
- Assumption: **At least one expert is perfect** (never makes mistakes)
- Maintain: $S_t \subseteq \{1, \dots, n\}$, initialized $S_1 = \{1, \dots, n\}$

For $t = 1, 2, \dots, T$:

1. Predict: $\hat{y}_t = \text{majority } \{\hat{y}_t^{(i)} : i \in S_t\}$
2. Observe: true outcome y_t
3. Update: $S_{t+1} = \{i \in S_t : \hat{y}_t^{(i)} = y_t\}$ (remove all wrong experts)

Theorem: Halving Mistake Bound (Perfect Expert Case)

If a perfect expert exists, the Halving Algorithm makes at most $\lceil \log_2 n \rceil$ mistakes.

Proof. Let M be the total number of mistakes made by the algorithm over T rounds.

Claim: Each time the algorithm makes a mistake, $|S_t|$ is at least halved.

Proof of claim:

- Algorithm predicts $\hat{y}_t$ by majority vote among S_t
- Algorithm makes mistake: $\hat{y}_t \neq y_t$
- This means majority of experts in S_t predicted $\hat{y}_t$ (wrong)
- Therefore, at least $\frac{|S_t|}{2}$ experts in S_t were wrong
- These experts are removed: $|S_{t+1}| \leq |S_t| - \frac{|S_t|}{2} = \frac{|S_t|}{2}$

Consequence after M mistakes:

- Start: $|S_1| = n$

- After 1st mistake: $|S| \leq \frac{n}{2}$
- After 2nd mistake: $|S| \leq \frac{n}{4}$
- After M mistakes: $|S| \leq \frac{n}{2^M}$

Perfect expert constraint:

- By assumption, at least one expert is perfect (never wrong)
- This expert is never removed: $|S_t| \geq 1$ for all t

Combining:

- $\frac{n}{2^M} \geq 1$
- $n \geq 2^M$
- $\log_2 n \geq M$
- Therefore: $M \leq \lceil \log_2 n \rceil$



Example: Halving with Perfect Expert (n=8)

Suppose $n = 8$ experts, and expert 1 is perfect (never wrong).

Round 1:

- $S_1 = \{1, 2, 3, 4, 5, 6, 7, 8\}$
- Suppose 5 experts predict 0, 3 predict 1
- Majority: predict 0
- Outcome: $y_1 = 1$ (mistake)
- Remove 5 experts who predicted 0 (expert 1 predicted 1, so safe)
- $|S_2| = 3$

Round 2:

- $S_2 = \{1, 6, 7\}$ (expert 1 + two others)
- Suppose expert 1 predicts 0, experts 6 and 7 predict 1
- Majority: predict 1
- Outcome: $y_2 = 0$ (mistake)
- Remove experts 6, 7 who predicted 1
- $|S_3| = \{1\}$ (only perfect expert remains)

Round 3 onwards:

- Only expert 1 remains (perfect)
- Algorithm follows expert 1: never wrong again

Total mistakes: $2 = \lceil \log_2 8 \rceil - 1 \leq \lceil \log_2 8 \rceil$ (bound holds tightly)

Why the proof fails without perfect expert:

The proof requires “ $|S_t| \geq 1$ for all t ” (line: “perfect expert is never removed”). If no perfect expert exists, this fails:

- Eventually all experts make mistakes
- All experts get eliminated

- $|S_t| = 0$ for some t
- Algorithm becomes undefined

The bound $M \leq \log_2 n$ is not just loose—it's **inapplicable** without the perfect expert assumption.

Lessons from Halving's Failure

Key Takeaway

What we learned from the Halving collapse:

1. **Logarithmic bounds are impossible without perfect experts.** The $O(\log n)$ mistake bound is a mathematical artifact of the perfect expert assumption. In realistic settings where all experts are imperfect, we cannot achieve logarithmic scaling.
2. **Hard elimination is too aggressive.** Removing experts permanently after one mistake prevents recovery. In adversarial settings, even good experts make occasional mistakes. We need soft downweighting, not hard elimination.
3. **The algorithm can fail catastrophically, not just perform poorly.** Halving doesn't gracefully degrade—it crashes when $S_t = \emptyset$. Robust algorithms must handle all scenarios, even worst-case adversarial sequences.
4. **Trade bound quality for robustness.** Weighted Majority achieves $O(\sqrt{T})$ regret instead of $O(\log n)$ mistakes, but works in all settings. The worse bound buys us universal applicability.
5. **Soft vs hard elimination appears throughout ML:**
 - Hard elimination: Halving, decision tree pruning (remove subtrees)
 - Soft elimination: Weighted Majority, neural networks (downweight, never remove)
 - Robustness generally favors soft methods

Forward Connection: Weighted Majority Algorithm

The failure of Halving motivates our next algorithm: **Weighted Majority**.

Key modifications:

- Replace binary set membership $i \in S_t$ with continuous weights $w_t(i) \in (0, \infty)$
- Replace hard elimination with multiplicative decay: $w_{t+1}(i) = (1 - \eta)w_t(i)$ when expert i is wrong
- Replace majority vote with weighted vote: $\hat{y}_t = \operatorname{argmax}_y \sum_{i: \hat{y}_t^{(i)} = y} w_t(i)$

What we gain:

- Algorithm never crashes: weights always positive
- Handles imperfect experts gracefully
- Provable regret bound: $O(\sqrt{T})$ without assumptions

What we lose:

- Worse bound: $O(\sqrt{T})$ vs $O(\log n)$
- Need to tune learning rate η
- More complex analysis (potential functions instead of simple counting)

But this trade is worthwhile: we move from theoretical curiosity (Halving) to practical algorithm (Weighted Majority).

→ **Week 8 Preview: Weighted Majority Algorithm**

Section 3 will develop Weighted Majority Algorithm, prove its $O(\sqrt{T})$ regret bound, and show why soft elimination via multiplicative weights is the key to robust online learning.

 Confidence Check

Self-Assessment: Before moving to Weighted Majority, check your understanding:

Rate your confidence (1-5):

- ☐ I can explain why Halving makes at most $\log_2 n$ mistakes under perfect expert assumption
- ☐ I understand why Halving fails completely without a perfect expert (set becomes empty)
- ☐ I can construct a concrete example where Halving crashes (all experts eliminated)
- ☐ I see why soft elimination (downweighting) is more robust than hard elimination
- ☐ I understand the trade-off: $O(\log n)$ with strong assumptions vs $O(\sqrt{T})$ with no assumptions

If 3 or below on any item: Review the Crisis box and the numerical example with $n = 100$.

Key question to ponder: In the concrete example ($n=100$, round 6), we eliminated the last remaining expert who made their first mistake. Was this the best expert overall? How many mistakes had they made? Could we have kept following them if we used weights instead of elimination?

DSAA3073: Theories in Data Science

Week 8: Section 3: Weighted Majority Algorithm (Cognitive Journey)

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-07-15

Explorative projection

The Weighted Majority Algorithm

We've seen that Halving achieves logarithmic mistakes but catastrophically fails without a perfect expert. Now we fix this with a crucial insight: instead of eliminating experts entirely, we can **downweight** them.

🧐 Student Question

From the class: “We fixed Halving’s crash by using soft elimination—reducing weights instead of removing experts. Now that all experts can contribute, can we approach the best expert’s performance? If the best expert makes M^* mistakes, we should make approximately M^* mistakes too, right?”

This seems like the natural goal: with weighted majority, we’re tracking which experts are better over time. The weights should converge to favor the best expert, so our performance should approach theirs.

The Natural Approach: Multiplicative Weight Decay

Strategy: Keep all experts but reduce influence when they’re wrong.

Definition: Weighted Majority Algorithm

Parameters: Learning rate $\eta \in (0, \frac{1}{2}]$

Initialization: $w_1(i) = 1$ for all experts $i \in \{1, \dots, n\}$

For each round $t = 1, \dots, T$:

1. **Predict:** Follow weighted majority

$$\text{Predict 0 if } \sum_{i: \text{predict 0}} w_t(i) \geq \sum_{i: \text{predict 1}} w_t(i) \text{ , else predict 1} \quad (11)$$

2. **Observe:** True outcome and which experts were correct/wrong
3. **Update:** For each expert i , multiply weight by $(1 - \eta)$ if wrong, keep unchanged if correct:

$$w_{t+1}(i) = \begin{cases} (1 - \eta)w_t(i) & \text{if expert } i \text{ was wrong} \\ w_t(i) & \text{if expert } i \text{ was correct} \end{cases} \quad (12)$$

Intuition:

- Wrong experts: multiply weight by $(1 - \eta) < 1 \rightarrow$ weight decreases
- Correct experts: keep weight unchanged $\rightarrow$ relative influence increases
- Over time: better experts accumulate more weight

This seems perfect! No crashes, smooth weight updates, and we track expert performance.

Numerical Failure: The Factor-2 Wall

Let's test this with a simple case.

Setup:

- $n = 2$ experts
- Expert 1: always predicts 0
- Expert 2: always predicts 1
- $T = 100$ rounds
- $\eta = 0.1$ (small learning rate)

Initial state:

- Weights: $w_1(1) = 1, w_1(2) = 1$
- Sum: $1 + 1 = 2$
- Weighted majority: Tie $\rightarrow$ predict 0 (or 1, doesn't matter)

What we expect: After many rounds, weights converge to favor the better expert. If expert 1 is better (outcome is 0 more often), we should make approximately M^* mistakes.

What actually happens:

Round	w(1)	w(2)	Outcome	What Happens
1	1.00	1.00	Adversary sees tie	Makes majority wrong $\rightarrow$ say outcome = 1
2	0.90	1.00	Majority predicts 1	Adversary chooses 0
3	0.90	0.90	Tie again	Adversary chooses 1
4	0.81	0.90	Majority predicts 1	Adversary chooses 0
5	0.81	0.81	Tie again	Adversary chooses 1

Pattern emerges: Adversary makes weighted majority wrong **every single time!**

After $T = 100$ rounds:

- Algorithm makes $M = 100$ mistakes
- Best expert (say expert 1) makes $M^* = 50$ mistakes
- Ratio: $\frac{M}{M^*} = \frac{100}{50} = 2.0$

⚠ Crisis

The Deterministic Trap: Factor-2 Wall

Setup: Two experts, deterministic Weighted Majority with any η

Expected: “Weights converge $\rightarrow$ performance approaches M^* ”

Reality:

- Algorithm: $M = 100$ mistakes
- Best expert: $M^* = 50$ mistakes
- Ratio: $\frac{M}{M^*} = 2.0$ forever

The gap: Expected “approaching M^* ” but got “ $2M^*$ permanently”

Shocking discovery: No tuning of η helps! The factor of 2 is a fundamental barrier for deterministic algorithms.

Why Does This Happen?

The adversary’s strategy:

1. Observe the weighted majority’s prediction (deterministic $\rightarrow$ predictable)
2. Choose outcome to make that prediction wrong
3. Weights adjust, but prediction is still deterministic next round
4. Repeat

Key insight: With two experts always disagreeing, one must be right and one wrong each round. The adversary ensures the weighted majority is always the wrong one.

Does Learning Rate Help?

Let’s try different η values:

Attempt 1: Large $\eta = 0.5$ (aggressive learning)

Intuition: “Large η means faster weight updates $\rightarrow$ faster convergence $\rightarrow$ better performance”

Scenario	$\eta = 0.1$	$\eta = 0.5$
Best expert M^*	50	50
Algorithm M	100	100
Ratio $\frac{M}{M^*}$	2.0	2.0

Result: Factor of 2 remains! Larger η doesn’t help.

Attempt 2: Tiny $\eta = 0.01$ (conservative learning)

Intuition: “Small η means gradual updates $\rightarrow$ more stable $\rightarrow$ better long-term performance”

Result: Still factor of 2!

⚠ Crisis

The Learning Rate Catastrophe

Intuition from convex optimization suggests: “Larger learning rate $\rightarrow$ faster convergence”

Testing with $n = 1000$, $M^* = 100$:

- $\eta = 0.9$ (large): Bound ≈ 395 mistakes
- $\eta = 0.26$ (optimal): Bound ≈ 305 mistakes
- $\eta = 0.01$ (tiny): Bound ≈ 1480 mistakes

Discovery: Mistake bound has form $M \leq 2(1 + \eta)M^* + \frac{2 \ln n}{\eta}$

- Large η : $\frac{2 \ln n}{\eta}$ term is small, but $2(1 + \eta)M^*$ is large
- Small η : $2(1 + \eta)M^* \approx 2M^*$, but $\frac{2 \ln n}{\eta}$ explodes
- Optimal: $\eta = \sqrt{\frac{\ln n}{M^*}}$ balances both terms

But: Even at optimal η , the factor of 2 remains!

The Formal Regret Bound

Theorem: Weighted Majority Mistake Bound

Let M^* be the number of mistakes made by the best expert. The Weighted Majority algorithm makes at most:

$$M \leq 2(1 + \eta)M^* + \frac{2 \ln n}{\eta} \quad (13)$$

mistakes.

Optimal choice: $\eta = \sqrt{\frac{\ln n}{M^*}}$ gives:

$$M \leq 2\sqrt{M^* \ln n} + 2M^* \quad (14)$$

The factor of 2: Notice the “2” multiplying $(1 + \eta)M^*$ and the “2” in front of $\sqrt{M^* \ln n}$. This is not a loose bound—it’s fundamental.

The Factor-2 is Unavoidable

Theorem (Informal): Any deterministic algorithm must have factor ≥ 2 ratio to best expert in adversarial setting.

Proof idea: Consider $n = 2$ experts that always disagree. In every round:

- One expert is right, one is wrong (they disagree on outcome)
- Best expert: right at least $\frac{T}{2}$ times $\rightarrow M^* \leq \frac{T}{2}$
- Deterministic algorithm: adversary sees prediction before outcome
- Adversary can always make algorithm wrong $\rightarrow M = T$

- Ratio: $\frac{M}{M^*} \geq \frac{T}{2} = 2$

This is not an artifact of our algorithm—it's fundamental to determinism!

Comparison

Halving Algorithm:

- Mistakes: $O(\log n)$
- Requirement: Perfect expert exists
- Failure mode: Crashes without perfect expert

Weighted Majority:

- Mistakes: $O(M^*)$
- Requirement: None (works with all imperfect experts)
- Limitation: Factor-2 barrier (deterministic)

The tradeoff:

- Halving: Amazing bound, catastrophic failure
- Weighted Majority: Robust, but factor-2 wall

Can we break the factor-2 barrier? Yes—with randomization!

The Weighted Majority algorithm is a major step forward: it works without perfect experts. But the factor-2 barrier reveals a fundamental limitation of determinism. The resolution requires a new ingredient: randomization.

Looking ahead: Randomized Weighted Majority will sample experts according to weights instead of deterministic voting. This private randomness prevents adversarial exploitation and achieves regret $O(\sqrt{T})$ without the factor-2.

DSAA3073: Theories in Data Science

Week 8: Section 4: Randomized Weighted Majority (Cognitive Journey)

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-07-15

Explorative projection

Randomized Weighted Majority: Breaking the Factor-2 Barrier

We've hit a fundamental wall: deterministic Weighted Majority cannot do better than $2M^*$. The factor of 2 is provably unavoidable. But there's a way out: randomization.

Student Question

From the class: “The factor-2 barrier happens because the adversary can predict our deterministic choice. But what if instead of always following the weighted majority, we **sample** randomly according to the weights? Then the adversary can't predict which expert we'll follow!”

This seems clever: when weights are $(0.6, 0.4)$, flip a biased coin—60% chance follow expert 1, 40% chance follow expert 2. The adversary sees the weights but not the coin flip result.

Question: Does this actually eliminate the factor of 2?

The Natural Approach: Deterministic Tie-Breaking

Before jumping to full randomization, let's try simpler “pseudo-random” approaches.

Idea 1: Alternation

When weights are equal (or nearly equal), alternate between experts:

- Round 1: Follow expert 1
- Round 2: Follow expert 2
- Round 3: Follow expert 1
- ...

Intuition: This “spreads out” the choices and avoids always following the same expert.

Seems reasonable! It's deterministic but not constant.

Numerical Failure: Adversary Exploits Patterns

Let's test alternation.

Setup:

- $n = 2$ experts (expert 1, expert 2)
- Both start with equal weights: $w(1) = w(2) = 1$
- Algorithm: alternate between experts when tied

What happens:

Round	Algorithm Follows	Outcome	Result	New Weights
-------	-------------------	---------	--------	-------------

1	Expert 1 (start)	Adversary: 1	Wrong	$w(1)=0.9, w(2)=1$
2	Expert 2 (alternate)	Adversary: 0	Wrong	$w(1)=0.9, w(2)=0.9$
3	Expert 1 (alternate)	Adversary: 1	Wrong	$w(1)=0.81, w(2)=0.9$
4	Expert 2 (heavier)	Adversary: 0	Wrong	$w(1)=0.81, w(2)=0.81$
5	Expert 1 (alternate)	Adversary: 1	Wrong	$w(1)=0.73, w(2)=0.81$

Pattern: Adversary observes the deterministic pattern and makes it wrong every time!

After $T = 100$ rounds:

- Algorithm: $M = 100$ mistakes (100% error rate!)
- Best expert: $M^* = 50$ mistakes
- Ratio: $M/M^* = 2.0$ (or worse!)

Other deterministic rules?

- Round-robin: Same problem
- “Follow heavier weight, tie-break by smaller index”: Same problem
- Any deterministic function of weights: Same problem

⚠ Crisis

The Randomization Mystery: ALL Deterministic Rules Fail

Attempt 1: Alternation between experts

- Result: 100% error rate ($M = T$)

Attempt 2: Round-robin scheduling

- Result: Adversary exploits schedule $\rightarrow M/M^* = 2.0$

Attempt 3: Deterministic tie-breaking by index

- Result: Adversary predicts tie-break $\rightarrow M/M^* = 2.0$

The shocking discovery: It's not about **which** deterministic rule we use. ANY deterministic function $f(\text{weights}) \rightarrow \text{prediction}$ can be exploited!

Why? The adversary has complete information about:

- Current weights (deterministic from history)
- Algorithm's rule $f(\cdot)$ (knows the algorithm)
- Therefore: Can predict $f(\text{weights}) \rightarrow \text{make it wrong}$

The gap: Expected “clever tie-breaking helps” but got “all deterministic rules fail equally”

Why Any Deterministic Rule Fails

Formal argument:

1. Algorithm state at round t : weights $w_t = (w_t(1), \dots, w_t(n))$
2. Deterministic algorithm: prediction = $f(w_t)$ for some function f
3. Adversary's knowledge:
 - Knows all past outcomes $(y_1, \dots, y_{t-1})$ — they chose them!

- Knows algorithm's rule f
 - Can compute w_t from $(y_1, \dots, y_{t-1})$ and rule f
 - Therefore: Can compute $f(w_t) = \text{prediction at round } t$
4. Adversary strategy: Choose $y_t \neq f(w_t) \rightarrow$ algorithm is wrong

This works for ANY deterministic f !

The issue isn't the specific rule—it's determinism itself.

The Key Insight: Randomness is Private Information

Aha Moment

The Resolution: True Randomization Breaks Exploitation

The factor-2 barrier exists because adversary can predict our choice.

Solution: Don't **choose** deterministically—**sample** randomly!

Randomized Weighted Majority:

- Weights: $w_t(1) = 0.6$, $w_t(2) = 0.4$
- Compute probabilities: $p_t(1) = 0.6/(0.6+0.4) = 0.6$, $p_t(2) = 0.4$
- Flip biased coin (60%-40%)
- Follow expert according to coin flip result

Why this works:

- Adversary sees weights (public information)
- Adversary does NOT see coin flip result (private information)
- Must choose outcome y_t before knowing which expert we sampled
- **Cannot exploit because prediction is truly unpredictable!**

The Non-Anticipating Adversary Model

Formal model:

Oblivious adversary: Chooses all outcomes $(y_1, \dots, y_T)$ before game starts

- Cannot see any of our choices
- Easiest adversary

Non-anticipating adversary: At round t , chooses y_t after seeing:

- Our past actions (which experts we followed in rounds $1, \dots, t-1$)
- Current weights w_t (deterministic function of history)
- But NOT our current random coin flip

Fully adaptive adversary: Sees everything including current coin flip

- Against this adversary, no algorithm can do well
- Too strong—makes learning impossible

Randomized WM is designed for non-anticipating adversary.

🧐 Student Question

Confusion: “Isn’t the non-anticipating model unrealistic? In real life, can’t the adversary see our current choice?”

Answer: The model assumes we flip the coin after the adversary commits to an outcome, or use cryptographic randomness. This balance—strong enough to be challenging, weak enough to allow learning—is what makes the framework work.

The Randomized Weighted Majority Algorithm

Definition: Randomized Weighted Majority

Parameters: Learning rate $\eta \in (0, \frac{1}{2}]$

Initialization: $w_1(i) = 1$ for all experts $i \in \{1, \dots, n\}$

For each round $t = 1, \dots, T$:

1. **Compute total weight:**

$$\Phi_t = \sum_{j=1}^n w_{t(j)} \quad (15)$$

2. **Compute probabilities:**

$$p_{t(i)} = \frac{w_{t(i)}}{\Phi_t} \quad \text{for all } i \quad (16)$$

3. **Sample expert:** Draw $i_t \sim p_t$ (i.e., sample expert i with probability $p_{t(i)}$)
4. **Predict:** Follow expert i_t ’s prediction
5. **Observe:** True outcome and which experts were correct/wrong
6. **Update weights:** Multiply by $(1 - \eta)$ if wrong, keep unchanged if correct

$$w_{t+1}(i) = \begin{cases} (1 - \eta)w_t(i) & \text{if expert } i \text{ was wrong} \\ w_t(i) & \text{if expert } i \text{ was correct} \end{cases} \quad (17)$$

Key difference from deterministic: Step 3 samples instead of voting.

How Sampling Eliminates Factor-2

Two experts example:

- Weights: $w_t(1) = 0.6, w_t(2) = 0.4$
- Probabilities: $p_t(1) = 0.6, p_t(2) = 0.4$

Deterministic WM:

- Always follows expert 1 (heavier weight)
- Adversary makes expert 1 wrong $\rightarrow$ 100% error when balanced

Randomized WM:

- Follows expert 1 with 60% probability
- Follows expert 2 with 40% probability
- Expected mistakes: $0.6 \times P[\text{expert 1 wrong}] + 0.4 \times P[\text{expert 2 wrong}]$

If adversary makes outcome = 0 (expert 1 wrong, expert 2 right):

- Algorithm mistakes with probability 0.6 (when we sampled expert 1)

If adversary makes outcome = 1 (expert 1 right, expert 2 wrong):

- Algorithm mistakes with probability 0.4 (when we sampled expert 2)

Best adversary can do: Choose outcome to maximize mistake probability

- Choose 0: we make mistake with probability 0.6
- Choose 1: we make mistake with probability 0.4
- Adversary chooses 0 $\rightarrow$ expected mistakes = 0.6 per round

Deterministic would have: 1.0 mistakes per round (100% when adversary makes majority wrong)

Improvement: 0.6 vs 1.0 in balanced case!

The Regret Bound: Factor-2 Gone!

Theorem: Randomized Weighted Majority Regret Bound

Against a non-anticipating adversary, the expected regret satisfies:

$$\mathbb{E}[R_T] = \mathbb{E} \left[\sum_{t=1}^T \ell_{t(\text{algo})} \right] - \min_{i=1}^n \sum_{t=1}^T \ell_{t(i)} \leq (1 + \eta)M^* + \frac{\ln n}{\eta} \quad (18)$$

Optimal choice: $\eta = \sqrt{\frac{\ln n}{T}}$ gives:

$$\mathbb{E}[R_T] \leq 2\sqrt{T \ln n} \quad (19)$$

Compare to deterministic:

- Deterministic: $M \leq 2(1 + \eta)M^* + \frac{2 \ln n}{\eta}$ has factor of 2
- Randomized: $\mathbb{E}[R_T] \leq (1 + \eta)M^* + \frac{\ln n}{\eta}$ **NO factor of 2!**

Both the 2 multiplying $(1 + \eta)$ and the 2 in $(2 \ln n)$ are gone!

Proof Sketch

The proof uses the same potential function method but analyzes expected loss:

Key insight: Expected loss in round t is the weighted average: $\mathbb{E}[\ell_t(\text{algo})] = \sum_i p_t(i) \ell_t(i)$, not worst-case. This gives tighter potential decrease: $\mathbb{E}(\Phi_{T+1}) \leq n \exp(-\eta \sum_t \mathbb{E}[\ell_t])$. Combined with $\Phi_{T+1} \geq \exp(-\eta(1 + \eta)M^*)$, we get $\sum_t \mathbb{E}[\ell_t] \leq (1 + \eta)M^* + \frac{\ln n}{\eta}$ with **no factor of 2**.

Setting $\eta = \sqrt{\frac{\ln n}{T}}$ yields $\mathbb{E}[R_T] \leq 2\sqrt{T \ln n}$.

Numerical Verification

For $n = 2$ experts, $T = 100$, $M^* = 50$:

- Deterministic WM: 100 mistakes (factor-2 barrier)
- Randomized WM: ≈ 67 expected mistakes (33% improvement)

The table below shows the $\sqrt{T}$ scaling advantage grows with T :

T	Deterministic	Randomized E(M)	Improvement
100	100	67	33%
1,000	1,000	553	45%
10,000	10,000	5,266	47%

As $T \rightarrow \infty$, the ratio approaches 1! The regret is sublinear while mistakes are linear.

Why Randomization Works: Game Theory Perspective

Deterministic algorithm: Pure strategy (always play the same action given state)

- Exploitable by adversary who knows the strategy

Randomized algorithm: Mixed strategy (probability distribution over actions)

- Adversary must choose outcome without knowing which action we sampled
- This is a **game** between learner and adversary
- Mixed strategies prevent exploitation!

Connection to minimax theorem:

- Learner wants to minimize loss
- Adversary wants to maximize learner's loss
- Randomization allows learner to achieve minimax optimal strategy

 Comparison**Halving Algorithm:**

- Bound: $O(\log n)$ mistakes
- Assumption: Perfect expert exists
- Failure: Crashes without perfect expert

Weighted Majority (Deterministic):

- Bound: $O(M^*)$ mistakes, but factor-2
- Assumption: None
- Limitation: $2M^*$ barrier is unavoidable

Randomized Weighted Majority:

- Bound: $O(\sqrt{T})$ regret
- Assumption: Non-anticipating adversary
- Achievement: **Factor-2 eliminated!**

The progression:

- Halving: Eliminate experts $\rightarrow$ crashes
- Deterministic WM: Downweight experts $\rightarrow$ factor-2
- Randomized WM: Sample experts $\rightarrow$ optimal!

Practical Implications

Key Takeaway

Using Randomized Weighted Majority

1. **When to use:** Randomization is available and acceptable
 - Most software environments support randomness
 - True randomness or cryptographic PRNGs
2. **How to implement:**
 - Maintain weights $w_t(i)$ for each expert
 - Compute $\Phi_t = \sum w_t(i)$
 - Sample expert i with probability $w_t \frac{i}{\Phi_t}$
 - Update weights: $w \leftarrow (1 - \eta)w$ for mistakes
3. **Learning rate:** Use $\eta = \sqrt{\frac{\ln n}{T}}$ if T known
 - If T unknown: use doubling trick or $\eta = \sqrt{\frac{\ln n}{t}}$ (time-varying)
4. **Expected vs. worst-case:**
 - Bound is on **expected** regret (average over randomness)
 - Individual runs may vary
 - High-probability bounds: Add $O\left(\sqrt{T \log\left(\frac{1}{\delta}\right)}\right)$ term for probability $1 - \delta$
5. **Practical performance:**
 - $n = 100$ experts, $T = 1000$ rounds
 - Regret $\approx 2\sqrt{1000} \times 4.6 \approx 135$
 - Much better than factor-2 of deterministic
6. **When randomization isn't possible:**
 - Embedded systems without RNG
 - Deterministic reproducibility required
 - Use deterministic WM and accept factor-2

The Non-Anticipating Model: Is It Realistic?

🧐 Student Question

Confusion: “In stock trading, after I place my order, the market responds. They see my action. Isn’t that a fully adaptive adversary?”

Clarification:

The timing model:

1. We compute weights w_t (adversary can observe—they’re deterministic from history)
2. We flip coin privately, sample expert i_t (adversary CANNOT observe)
3. We follow expert i_t ’s recommendation (NOW adversary sees our action)
4. Adversary chooses outcome y_t

Wait—doesn’t step 3 happen before step 4?

Two scenarios:

Scenario A: We reveal action before outcome

- Our action: place order (visible)
- Adversary: sees order, responds with price movement
- This IS fully adaptive! Randomization doesn’t help here.

Scenario B: Outcome determined independently

- Our action: internal decision
- Outcome: external event (stock price set by market dynamics, not just us)
- Our action doesn’t directly determine outcome
- This IS non-anticipating!

The model applies when:

- Outcomes are not solely determined by our actions
- Our randomness is private (adversary doesn’t see coin flips)
- We’re one small player among many (market doesn’t react to us alone)

Summary: Randomness as a Defense

 Comparison**Before Understanding Randomization:**

- “Factor-2 seems like a loose bound—maybe we can tighten it”
- “Deterministic tie-breaking rules might help”
- “Alternation or round-robin should spread out mistakes”

After Understanding Randomization:

- “Factor-2 is fundamental to determinism”
- “ANY deterministic rule can be exploited”
- “True randomness is the only way to break the barrier”

The Key Insight:

- Determinism = Predictability = Exploitability
- Randomization = Privacy = Unexploitability
- This is a game-theoretic necessity, not an algorithmic trick

The Achievement:

- Eliminated factor-2: $2M^* \rightarrow M^* + O(\sqrt{T})$
- Optimal regret: $O(\sqrt{T})$ is best possible in adversarial setting
- Robust: Works for all imperfect experts

Randomized Weighted Majority achieves the best of all worlds: no perfect expert assumption, no crashes, and optimal $O(\sqrt{T})$ regret. The factor-2 barrier is not just reduced—it’s completely eliminated through the power of randomization.

Looking ahead: This algorithm generalizes to continuous losses (not just binary mistakes) and connects to AdaBoost, online gradient descent, and game theory—all unified through the Multiplicative Weights Update framework.

DSAA3073: Theories in Data Science

Week 8: Section 5: Multiplicative Weights Update Framework (Cognitive Journey)

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-07-15

Explorative projection

Multiplicative Weights Update: Unifying the Landscape

We've journeyed through three algorithms—Halving, Weighted Majority, and Randomized Weighted Majority—each fixing a catastrophic failure of its predecessor. Now we face a broader question: are these specific solutions to specific problems, or instances of something deeper?

The Question: How Do These Problems Relate?

So far, we've tackled the **expert advice problem with binary outcomes**:

- Round t : Each expert makes binary prediction (0 or 1)
- Outcome reveals which experts were correct/wrong
- Loss is binary: $m_t(i) \in \{0, 1\}$

🤔 Student Question

From the class: “All our algorithms handle binary mistakes (correct/wrong). But real problems have continuous losses—stock returns, prediction errors, click-through rates. Are these completely different problems requiring different algorithms?”

It seems like we'd need separate theory for:

- Binary expert advice (what we've studied)
- Portfolio allocation (continuous returns)
- Online gradient descent (convex optimization)
- Game theory (mixed strategies)
- Boosting (weighted training examples)

Domain separation table:

Domain	Loss Type	What We Predict	Application
Expert advice	Binary $\{0, 1\}$	Which expert to follow	Stock prediction
Portfolio	Continuous $[0, 1]$	Budget allocation	Investment
Online GD	Continuous $[0, \infty)$	Point in space	Convex optimization
Boosting	Binary $\{0, 1\}$	Example weights	Classification
Game theory	Continuous $[0, 1]$	Mixed strategy	Equilibrium

These seem like five different problems from five different communities. Do we need five separate analyses?

The Fragmentation Crisis

🧐 Student Question

The natural intuition: “Continuous losses provide more information than binary mistakes. A loss of 0.3 tells us more than just ‘wrong’. So continuous losses should give **better** regret bounds than $O(\sqrt{T})$, right?”

This seems reasonable because:

- More information $\rightarrow$ better learning
- Binary is just a special case of continuous
- Smoother loss landscape should be easier to optimize

Let’s test this intuition with portfolio allocation.

Concrete Portfolio Example

Setup:

- $n = 10$ stocks (experts)
- $T = 1000$ days
- Each day: stock i has return $r_t(i) \in [-1, 1]$
- Convert to loss: $m_t(i) = (1 - r_t(i))/2 \in [0, 1]$
- Best single stock total loss: $L^* = 200$

Natural expectation: Continuous losses are “smoother” than binary, so maybe we get $O(\log T)$ regret instead of $O(\sqrt{T})$?

Apply MWU algorithm:

- Initialize: $w_1(i) = 1$ for all stocks
- Each day: allocate $p_t(i) = w_t(i) \frac{e^{-\eta m_t(i)}}{\sum_j w_t(j) e^{-\eta m_t(j)}}$
- Update: $w_{t+1}(i) = w_t(i) e^{-\eta m_t(i)}$
- Choose $\eta = \sqrt{\frac{\ln n}{T}} = \sqrt{\frac{2.3}{1000}} \approx 0.048$

Result:

- Regret bound: $2\sqrt{T \ln n} = 2\sqrt{1000 \times 2.3} \approx 96$
- Average per-day regret: $\frac{96}{1000} = 0.096$

Still $O(\sqrt{T})$! Not better than binary case!

⚠ Crisis

The Continuous Loss Illusion

Expected: Continuous losses $[0,1]$ provide more information $\rightarrow$ better than $O(\sqrt{T})$ regret

Reality: Regret is still $2\sqrt{T \ln n} \approx 96$

The gap:

- Binary mistakes: Regret = $O(\sqrt{T})$
- Continuous losses: Regret = $O(\sqrt{T})$
- No improvement!

Is this a coincidence? Five different domains (experts, portfolio, OGD, boosting, games) all achieve $O(\sqrt{T})$. What's the common structure?

The Puzzle Deepens

Let's check if this $O(\sqrt{T})$ rate appears elsewhere:

Problem	Loss Range	Regret	Source
Expert advice (binary)	$\{0,1\}$	$O(\sqrt{T})$	Randomized WM
Portfolio allocation	$[0,1]$	$O(\sqrt{T})$	MWU
Online gradient descent	$[0, \infty)$	$O(\sqrt{T})$	OGD analysis
AdaBoost (training)	$\{0,1\}$	$O(\sqrt{T})$	Week 6
Game theory	$[0,1]$	$O(\sqrt{T})$	Equilibrium computation

The mystery: All five domains achieve the same $O(\sqrt{T})$ rate. Different communities discovered this independently. Is there a unifying principle?

The Surprising Discovery

💡 Aha Moment

The Unification: They're All the Same Algorithm!

All five domains use the **identical multiplicative weight update**:

$$w_{t+1}(i) = w_{t(i)} \cdot (1 - \eta m_{t(i)}) \quad (20)$$

What varies is only the **interpretation** of $m_t(i)$:

- Expert advice: $m_t(i)$ = mistake indicator
- Portfolio: $m_t(i)$ = normalized loss
- OGD: $m_t(i)$ = gradient component
- AdaBoost: $m_t(i)$ = $\exp(-\alpha y h(x))$
- Game theory: $m_t(i)$ = payoff

Same formula, five applications!

The Unified Framework: MWU

Definition: Multiplicative Weights Update (MWU)

Given: n experts, T rounds

Initialization: $w_1(i) = 1$ for all i in $\{1, \dots, n\}$

For each round $t = 1, \dots, T$:

1. **Choose distribution:**

$$p_{t(i)} = \frac{w_{t(i)}}{\Phi_t} \quad \text{where} \quad \Phi_t = \sum_{j=1}^n w_{t(j)} \quad (21)$$

2. **Observe:** Loss vector $m_t = (m_t(1), \dots, m_t(n))$ with $m_t(i) \in [0,1]$

3. **Update weights:**

$$w_{t+1}(i) = w_{t(i)} \cdot (1 - \eta m_{t(i)}) \quad (22)$$

Learner's expected loss: $\langle p_t, m_t \rangle = \sum_i p_t(i) m_t(i)$

Key insight: The algorithm doesn't care what $m_t(i)$ represents. As long as losses are in $[0,1]$, the same analysis applies.

Why This Unification Is Powerful

Benefit 1: Single Analysis Covers All Cases

Theorem: MWU Regret Bound

With learning rate $\eta \in (0, \frac{1}{2}]$ and losses $m_t(i) \in [0, 1]$:

$$\text{Regret}_T = \sum_{t=1}^T \langle p_t, m_t \rangle - \min_{i \in [n]} \sum_{t=1}^T m_t(i) \leq \frac{\ln n}{\eta} + \eta T \quad (23)$$

Optimal choice $\eta = \sqrt{\frac{\ln n}{T}}$ gives:

$$\text{Regret}_T \leq 2\sqrt{T \ln n} \quad (24)$$

Proof: Identical potential function analysis as Randomized WM:

- Define potential function over all weights
- Show potential decreases by expected loss
- Bound from below using best expert
- Combine to get regret bound

Key point: The proof works for **any** losses in the interval from 0 to 1, not just binary!

Benefit 2: The Square Root Rate Is Fundamental

The square root of T rate appears across all domains because:

1. **Upper bound:** MWU achieves square root regret universally
2. **Lower bound:** Any online learning algorithm must have at least $\sqrt{T}$ regret in adversarial setting (provable)
3. **Therefore:** $\sqrt{T}$ is optimal—not an artifact of binary losses or specific problems

 Key Takeaway**Why Square Root Growth Is Unavoidable**

The square root rate comes from balancing two terms:

- Exploration cost: $\frac{\ln n}{\eta}$ suggests using large η
- Exploitation cost: ηT suggests using small η

Optimal tradeoff: $\eta = \sqrt{\frac{\ln n}{T}}$ makes both terms proportional to $\sqrt{T}$.

This tradeoff is fundamental to adversarial learning, not specific to any domain.

Benefit 3: Cross-Domain Insights

Understanding MWU in one domain illuminates others:

From expert advice $\rightarrow$ portfolio:

- Expert predictions = Stock allocations
- Mistake rate = Portfolio loss
- Best expert = Best single stock

From portfolio $\rightarrow$ online gradient descent:

- Stocks = Directions in space
- Returns = Negative gradients
- Allocation = Step in direction

From OGD $\rightarrow$ AdaBoost:

- Directions = Training examples
- Gradients = Example-specific losses
- Steps = Weight updates

Circular understanding: Each domain provides intuition for the others!

Comparison Table: One Algorithm, Many Faces

The MWU framework applies across domains with identical analysis:

Domain	What is $m_t(i)$?	Regret Compares To
Expert advice	1 if expert i wrong, 0 else	Best single expert
Portfolio	Normalized loss of stock i	Best single stock
Online GD	Loss gradient in direction i	Best fixed point
AdaBoost	$\exp(-\alpha y_i h(x_i))$	Best hypothesis sequence
Game theory	Payoff against action i	Best fixed strategy

Key insight: Same weight update $w_{t+1}(i) = w_{t(i)} \cdot (1 - \eta m_{t(i)})$, identical regret bound $O(\sqrt{T \ln n})$ across all domains.

Connections to Other Algorithms

Connection 1: AdaBoost (Week 6)

Recall AdaBoost's weight update with exponential reweighting.

This is MWU in disguise!

Setting:

- “Experts” = Training examples
- “Loss” = exponential loss (continuous in $[0, \infty)$)
- “Distribution” = Weights D_t over examples

Mapping:

- $w_t(i) \leftarrow D_t(i)$
- $m_t(i) \leftarrow$ measure of example i 's “difficulty”
- $p_t(i) \leftarrow$ distribution for weak learner training

Insight: AdaBoost is online learning over training examples! The weak learner adapts to the MWU-generated distribution.

Key Takeaway

AdaBoost = MWU + Weak Learning

Week 6 mystery solved: Why does AdaBoost use exponential weights?

Answer: Because it's performing MWU! The exponential weight update $w \leftarrow w(1 - \eta \text{loss})$ is optimal for adversarial online learning.

The connection:

- MWU learns which experts are best
- AdaBoost learns which examples are hard
- Same principle, different application

Connection 2: Online Gradient Descent

OGD updates by taking gradient steps—this is MWU where “experts” are directions in space, “losses” are gradient components, and “distribution” is the weighted combination of directions.

Connection 3: Game Theory

For computing Nash equilibria, players update strategies using MWU where “experts” are pure strategies and “losses” are payoffs.

The Complete Picture

Comparison

Before Understanding MWU:

- “Binary expert advice is one algorithm”
- “Portfolio allocation needs different techniques”
- “OGD, AdaBoost, game theory are separate”
- “Why does everything achieve $O(\sqrt{T})$?”

After Understanding MWU:

- “All use identical weight update: $w \leftarrow w(1 - \eta \text{loss})$ ”
- “Only interpretation of $m_t(i)$ varies”
- “Single analysis proves regret bound for all”
- “ $O(\sqrt{T})$ is fundamental to adversarial learning”

The Transformation: From: Five separate algorithms in five communities To: One unified principle with five applications

Summary Table: The MWU Landscape

Algorithm	Special Case	Regret
Halving	MWU with $\eta = 1$, perfect expert	$O(\log n)$
Weighted Majority	MWU with $\eta < 1$, binary losses, deterministic	$O\left((1 + \eta)M + \frac{\ln n}{\eta}\right)$
Randomized WM	MWU with binary losses, randomized	$O(\sqrt{T \ln n})$
MWU (general)	Losses in $[0,1]$	$O(\sqrt{T \ln n})$
Portfolio	MWU with return-based losses	$O(\sqrt{T \ln n})$
Online GD	MWU on simplex with gradient losses	$O(\sqrt{T})$
AdaBoost	MWU over examples	$O(\sqrt{T})$ training error

Reading the table: Each row is the same algorithm applied to different problem. The regret bound is identical (up to constants) except when additional structure (perfect expert, small M) improves the rate.

Why the $\sqrt{T}$ Barrier Is Fundamental

Two perspectives on why $O(\sqrt{T})$ cannot be improved:

Perspective 1: Information-Theoretic Lower Bound

Theorem: Any algorithm must have $\Omega(\sqrt{T})$ regret in adversarial setting.

Intuition:

- T rounds $\rightarrow \sqrt{T}$ bits of information about best expert
- Need $\Omega(\sqrt{T})$ regret to learn from this information
- No algorithm can beat this bound

Perspective 2: Exploration-Exploitation Tradeoff

Regret decomposes:

$$\text{Regret} = \underbrace{\frac{\ln n}{\eta}}_{\text{exploration cost}} + \underbrace{\eta T}_{\text{exploitation cost}} \quad (25)$$

- Small η : More exploration, less exploitation, so $\frac{\ln n}{\eta}$ is large
- Large η : Less exploration, more exploitation, so ηT is large
- Optimal: Balance at $\eta = \sqrt{\frac{\ln n}{T}}$ makes both terms proportional to $\sqrt{T}$

This tradeoff is unavoidable in adversarial settings without additional assumptions.

Practical Implications

Key Takeaway

Using MWU Effectively

1. **Recognize MWU instances** - If you have:
 - n options to choose from
 - Repeated decisions over time
 - Feedback on loss/performance
 → MWU likely applies!
2. **Set eta correctly** - Use $\eta = \sqrt{\frac{\ln n}{T}}$ if T known, or $\eta = \sqrt{\frac{\ln n}{t}}$ dynamically
3. **Normalize losses to [0,1]** - MWU requires bounded losses
4. **Interpret the regret bound** - $R_T = O(\sqrt{T})$ means:
 - After $T = 10000$: regret ≈ 100
 - Average per-round: 0.01 → vanishing
 - You approach best expert's performance
5. **Domain-specific tuning** - While core algorithm is the same:
 - Portfolio: May want risk-adjusted losses
 - OGD: May add regularization
 - AdaBoost: May use weighted sampling
6. **This extends beyond [0,1]** - For unbounded losses, use exponential weights:

$$w_{t+1}(i) = w_t(i) \exp(-\eta m_t(i)) \quad (26)$$

Looking Forward: Beyond MWU

The MWU framework opens doors to:

Extensions:

- Follow-The-Regularized-Leader (FTRL): Adds regularization for smoother updates
- Adaptive learning rates: η_t varies with observed losses
- Bandit settings: Only observe loss of chosen expert (not all experts)

Advanced topics (beyond this course):

- Internal regret: Compete with dynamic strategies
- Swap regret: Compete with best response mappings
- Contextual bandits: Expert losses depend on features

Real-world applications:

- A/B testing: Which website design performs best?
- Recommender systems: Which items to show users?
- Resource allocation: How to distribute compute/budget?

All build on the MWU foundation.

Final Reflection

We began Week 8 asking: “Can we learn without distributional assumptions?” The answer is yes—through online learning and regret minimization.

We discovered three algorithms:

1. Halving: $O(\log n)$ mistakes with perfect expert
2. Weighted Majority: $O((1 + \eta)M)$ with determinism
3. Randomized WM: $O(\sqrt{T})$ breaking the factor-2 barrier

Then we unified them all:

Multiplicative Weights Update—one algorithm, many applications, fundamental square root rate.

The journey revealed:

- **Adversarial learning is possible** (without i.i.d.)
- **Regret is the right metric** (compare to hindsight best)
- **Exponential weights are optimal** (not a heuristic)
- **$O(\sqrt{T})$ is fundamental** (exploration-exploitation tradeoff)
- **Cross-domain unification** (experts = portfolio = OGD = boosting)

This completes our understanding of how online learning works and why MWU is one of the most powerful algorithms in computer science.

Conclusion: The Online Learning Paradigm

We've journeyed through online learning from five perspectives:

The Five Insights

1. Adversarial Learning is Possible

- No distributional assumptions needed
- Regret compares to best fixed strategy
- Sublinear regret $\rightarrow$ vanishing average loss

2. Halving: Logarithmic with Perfect Expert

- $O(\log n)$ mistakes when perfect expert exists
- Catastrophically fails without one
- Led to soft elimination idea

3. Weighted Majority: The Factor-2 Barrier

- Soft weights prevent crashes
- But determinism hits provable factor-2 wall
- No tuning of η helps

4. Randomization Breaks the Barrier

- True randomness is private information
- Non-anticipating adversary cannot exploit
- Achieves optimal $O(\sqrt{T})$ regret

5. MWU Unifies Everything

- One algorithm: $w \leftarrow w(1-\eta m)$
- Five applications: experts, portfolio, OGD, AdaBoost, games
- $O(\sqrt{T})$ is fundamental to adversarial learning

The Unifying Thread

All algorithms use **multiplicative weight updates**:

- Halving: $w \leftarrow 0$ or w (hard elimination)
- Weighted Majority: $w \leftarrow (1-\eta)w$ or w (deterministic)
- Randomized WM: Sample from weights (randomized)
- MWU: $w \leftarrow w(1-\eta m)$ (continuous losses)

Same principle, increasing generality!

Practical Implications

🎯 Key Takeaway

Using Online Learning Effectively

1. **Recognize the setting:** Sequential decisions, adversarial outcomes
2. **Choose the right algorithm:**
 - Perfect expert exists? Use Halving
 - Binary losses, small n ? Use Randomized WM
 - Continuous losses? Use MWU
 - Convex optimization? Use Online GD
3. **Set η correctly:** $\eta = \sqrt{(\ln n)/T}$ is optimal
4. **Interpret regret:** $R_T = O(\sqrt{T})$ means average $\rightarrow 0$
5. **This extends everywhere:** Bandits, contextual learning, FTRL

Connections

To Week 6 (Boosting): AdaBoost is MWU over training examples

To Week 9 (Deep Learning): SGD is Online GD with mini-batches

To Game Theory: MWU computes Nash equilibria

Summary Table

Algorithm	Assumption	Regret	Key Property
Halving	Perfect expert	$O(\log n)$	Hard elimination
Weighted Majority	Deterministic	$O(M^*)$	Factor-2 barrier
Randomized WM	Randomized	$O(\sqrt{T})$	Breaks barrier
MWU	General losses	$O(\sqrt{T})$	Unifies all

The journey through online learning reveals a beautiful progression: each algorithm fixes a catastrophic failure of its predecessor, culminating in MWU—one of the most fundamental algorithms in computer science.